Field Hours

CSCE 669 Computational Optimization

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Polyhedra and linear objectives

You treat a combinatorial search as a linear program on a polyhedron. The feasible set is every vector x with A x <= b. A vertex is a point where enough inequalities are equalities to pin the coordinates. Maximize x + y subject to x + y <= 3, x <= 2, y <= 2, and both variables at least 0. The optimum value is 3, and every optimal point lies on the edge where x + y = 3.

You do not need every point of that edge. A linear objective on a nonempty bounded polyhedron attains its optimum at a vertex, so you can restrict attention to vertices. Here the vertices include (0,0), (2,0), (0,2), and the objective values are 0, 2, and 2. The edge from (1,2) to (2,1) is optimal, and both of those corners give value 3. The mechanism is the vertex claim, which lets you stop searching the interior.

A linear program is a polyhedron plus an objective.

Inequalities A x <= b

Vertices pin coordinates

Objective x + y

Optimum value is 3

Maximize x + y

1. Constraints: x + y <= 3, x <= 2, y <= 2, and both variables at least 0.

2. The point (2, 1) is feasible and scores 3. The point (2, 2) scores 4 but violates x + y <= 3.

3. Every optimal point lies on x + y = 3 between (1, 2) and (2, 1). Both of those corners score 3.

Problem 1. Is (0, 3) feasible for x + y <= 3, x <= 2, y <= 2, x and y at least 0? What value does (2, 1) have?

Answer. (0, 3) violates y <= 2, so it is not feasible. (2, 1) meets every inequality and scores 2 + 1 = 3.

Problem 2. Why may you look only at vertices for a linear objective on a nonempty bounded polyhedron?

Answer. The optimum is attained at a vertex, so no interior point beats every vertex.

Lecture 2 · 90 minutes

Flows, cuts, and three augmentations

You move flow on edges that have capacities. A feasible flow stays between 0 and the capacity on each edge and

balances at every vertex except the source and the sink. An augmenting path still has spare capacity on every edge. If the source has two outgoing edges of capacities 3 and 2, no flow exceeds 5. A cut of capacity 5 matching a flow of 5 proves optimality, because the max flow equals the min cut.

You reach that value 5 by three pushes. Use edges source to u of capacity 3, source to v of capacity 2, u to sink of capacity 2, v to sink of capacity 3, and u to v of capacity 1. Push 2 on source, u, sink. Push 2 on source, v, sink. Push 1 on source, u, v, sink. The cut around the source has capacity 3 + 2 = 5. Flow and cut match, so 5 is optimal.

Augment until the flow matches a cut.

Push 2 through u -> Push 2 through v -> Push 1 through u, v -> Match the source cut

Three pushes, value 5

1. Push 2 on source, u, sink. Push 2 on source, v, sink. Push 1 on source, u, v, sink.

2. The shipped value is 2 + 2 + 1 = 5.

3. The source cut has capacity 3 + 2 = 5. Equal numbers mean the flow is maximum and the cut is minimum.

Problem 1. After the first push of 2 along source, u, sink, what residual capacity remains on source to u, and on u to sink?

Answer. Source to u started at 3, so 1 remains. u to sink started at 2, so 0 remains.

Problem 2. Why does a feasible cut of capacity 5 prove that a feasible flow of value 5 is optimal?

Answer. No flow can exceed a cut, and the maximum flow equals the minimum cut. Matching numbers leave a gap of 0.

Lecture 3 · 90 minutes

Bipartite matching and Konig

You model an assignment as a matching in a bipartite graph. A matching is a set of edges that share no vertex. An augmenting path starts and ends at a free vertex and alternates between unused and used edges. Replacing used edges by unused edges along that path raises the matching size by 1. With 4 vertices on each side, a perfect matching has size 4, so you need 4 successful augmentations from the empty matching.

You stop when no augmenting path remains. In a bipartite graph, Konig's theorem says the maximum matching size equals the minimum vertex cover size. Take left vertices A and B and right vertices 1, 2, and 3, with edges A-1, A-2, B-2, and B-3. The matching A-1 together with B-3 has size 2. The set {A, B} is a cover of size 2. Equal numbers mean both answers are optimal.

Grow a matching along one alternating path.

1. Start at a free vertex

2. Alternate unused and used

3. Flip along the path

4. Size grows by 1

A matching and a cover of size 2

1. Edges are A-1, A-2, B-2, and B-3. Start from the empty matching.

2. Add A-1. Then B and 3 are free, so add B-3. The matching has size 2.

3. The cover {A, B} also has size 2. Konig's theorem says the matching is maximum.

Problem 1. On the same graph, is {A-2, B-3} a matching, and what is its size?

Answer. Yes. A-2 and B-3 share no vertex, so the size is 2, equal to the minimum cover.

Problem 2. You have a matching of size 3 in a bipartite graph with 4 vertices on each side. How many further augmentations produce a perfect matching?

Answer. A perfect matching has size 4, so you need 4 - 3 = 1 more successful augmentation.

Lecture 4 · 90 minutes

Matroids and greedy choice

You call a family of sets a matroid when two rules hold. Every subset of an independent set is independent. If one independent set is larger than another, some element of the larger one can be added to the smaller one without leaving the family. Forests in a graph form a graphic matroid. A forest on 6 vertices has at most 5 edges, and you may add any edge that does not close a cycle.

You optimize linear weights by sorting and adding. Order elements by weight, high to low, and add one when the set stays independent. On a uniform matroid of rank 2 with weights 6, 4, and 1, you take 6, then 4, and you reject 1. The total is 10. No independent set scores higher. The same exchange rule is Kruskal's algorithm when the independent sets are the forests of a graph.

Exchange is why greedy can commit.

Uniform rank 2

Weights 6, 4, and 1

At most 2 elements

Best total is 10

Greedy takes

Sort high to low

Take 6, then 4

Reject 1

Greedy on weights 6, 4, and 1

1. Rank 2 forbids every set of size 3. Sort the weights as 6, then 4, then 1.

2. Add 6. Add 4. The set of those two weights is independent and full.

3. Reject 1. The total is 10. The rival pair of weights 6 and 1 totals 7, and 4 and 1 totals 5.

Problem 1. Same uniform matroid of rank 2, with weights 9, 8, and 7. What does greedy return?

Answer. It takes 9 and 8, rejects 7, and the total is 17. That is the best pair.

Problem 2. A forest on 6 vertices already has 5 edges. May you add another edge and stay in the graphic matroid?

Answer. No. A forest on 6 vertices has at most 5 edges. One more edge closes a cycle.

Lecture 5 · 90 minutes

Integral vertices

You get integer optima from a linear program when the polyhedron is integral. A matrix is totally unimodular when every square submatrix has determinant 0, 1, or -1. If A is totally unimodular and b is integer, the vertices of the polyhedron defined by A x <= b and x >= 0 are integer. The vertex-edge incidence matrix of a bipartite graph has this property. That is why the bipartite matching linear program returns a 0-1 vector at a vertex.

You can see an integral vertex on a one-constraint program. Maximize x + y subject to x + y <= 1, with x and y at least 0. The vertices are (0,0), (1,0), and (0,1), and the optimum value is 1. The point (1/2, 1/2) is feasible and also has value 1, but it is not a vertex. An integral polyhedron may contain fractional points. The promise is that some optimum is a vertex, and that vertex has integer coordinates.

Integrality is a promise about vertices.

Totally unimodular matrix

Integer right-hand side

An integer optimum vertex

Fractional points may remain

Vertices of x + y <= 1

1. Maximize x + y subject to x + y <= 1, with x and y at least 0.

2. The vertices are (0,0), (1,0), and (0,1). Each is integer, and the best value is 1.

3. The midpoint (1/2, 1/2) is feasible and also scores 1, but it is not a vertex. Integrality does not ban that midpoint.

Problem 1. List the vertices of x + y <= 1 with x and y at least 0, and give the maximum of x + y.

Answer. The vertices are (0,0), (1,0), and (0,1). The maximum is 1.

Problem 2. Why does total unimodularity of the bipartite incidence matrix matter for matching?

Answer. With an integer right-hand side, every vertex is integer, so a vertex solution of the matching linear program is a 0-1 matching.

Lecture 6 · 90 minutes

Rounding a cover

You approximate vertex cover by rounding a linear program. For each edge the endpoint variables sum to at least 1, each variable lies between 0 and 1, and you minimize their sum. Set a variable to 1 when its value is at least 1/2, and to 0 otherwise. Every edge then has a selected endpoint, so the result is a cover. Each selected 1 is at most twice the fractional value it replaces, which is a factor of 2.

You watch that factor on a triangle. The linear program sets each of the 3 variables to 1/2 and scores 3/2. Rounding at 1/2 selects all 3 vertices. An integral optimum has size 2, since one vertex leaves an edge uncovered. Against the linear-program value the ratio is 3 divided by 3/2, which equals 2. Against the integral optimum the ratio on this graph is 3/2. The guarantee you prove for every graph is still 2.

Round a fractional cover up to an integral cover.

Solve the cover LP -> Threshold at 1/2 -> Keep those vertices -> Factor at most 2

Triangle, LP value 3/2

1. Each edge forces two variables to sum to at least 1. Setting each of the 3 variables to 1/2 scores 3/2.

2. Rounding at 1/2 selects all 3 vertices.

3. An integral optimum has size 2. The returned cover has size 3. The general argument still claims only a factor of 2.

Problem 1. The fractional optimum is 3/2 and rounding returns 3 vertices. What is the ratio to the fractional value?

Answer. 3 / (3/2) = 2.

Problem 2. An edge has fractional endpoints 0.7 and 0.3. Which endpoint does the 1/2 rule select, and is the edge covered?

Answer. It selects 0.7 and drops 0.3. The edge is covered because 0.7 is at least 1/2.

Lecture 7 · 90 minutes

Dual certificates

You certify optimality with a dual program. If the primal maximizes a linear function over A x <= b and x >= 0, the dual minimizes a linear function of nonnegative multipliers on those inequalities. Weak duality says every dual-feasible value is at least every primal-feasible value. Maximize 2x subject to x <= 4 and x >= 0. The primal optimum is 8. The dual minimizes 4y subject to y >= 2, and its optimum is also 8.

You treat a gap as unfinished work. A primal value of 6 and a dual value of 8 only pin the optimum to the interval from 6 to 8. Equal numbers are the certificate. On the program that maximizes 2x with x <= 4, the point x = 4 scores 8 and the dual point y = 2 scores 8. Both are optimal. The mechanism is one feasible dual solution, not a second search through primal vertices.

Equal primal and dual values close the gap.

Primal

Maximize 2x

Constraint x <= 4

Best value 8

Dual

Minimize 4y

Constraint y >= 2

Best value 8

Close the gap at 8

1. The primal point x = 4 is feasible and scores 2*4 = 8.

2. The dual point y = 2 is feasible and scores 4*2 = 8.

3. Weak duality says no primal value exceeds a dual value. Since both equal 8, both are optimal.

Problem 1. You hold a primal value of 6 and a dual value of 8. What do you know about the optimum?

Answer. It lies between 6 and 8. The gap is 8 - 6 = 2, so you cannot yet call either number optimal.

Problem 2. For maximize 2x subject to x <= 4, which primal and dual points certify optimality, and at what common value?

Answer. x = 4 and y = 2. Both score 8, so the optimum is 8.

Lecture 8 · 90 minutes

What you can now certify

You can now cast a combinatorial question as a polyhedron and say what that buys. A flow of 5 matched by a cut of 5 is optimal. A bipartite matching of size 2 matched by a cover of size 2 is optimal. A matroid greedy total of 10 is optimal by exchange. A totally unimodular matrix lets you trust an integer vertex. Without that property, rounding still yields a proved factor, such as 2 for vertex cover.

You can also stop a solver with a dual certificate. A primal value and a dual value that both equal 8 leave no gap. A primal value of 6 against a dual value of 8 leaves a gap of 2, so you do not claim optimality yet. The practice is to write the inequalities, name the algorithm that walks them, and attach either an exact identity, such as max flow equals min cut, or an explicit approximation factor.

What you can now attach to a feasible solution.

1. Write the inequalities

2. Run the algorithm

3. Match a cut or a dual

4. Or state the factor

A matching cut beside a rounded cover

1. A feasible flow has value 5 and the source cut has capacity 5, so the flow is optimal.

2. On the triangle, the cover linear program scores 3/2 and rounding returns 3 vertices.

3. The ratio of 3 to 3/2 is 2. You use a matching cut when you have one, and the factor when the vertex you want is not integral.

Problem 1. Combine max flow with weak duality. A flow of value 5 is feasible and a cut has capacity 7. Can you call the flow optimal, and what cut capacity would finish the proof?

Answer. No. Capacity 7 only shows the optimum is at most 7, the same way a dual value of 8 only upper-bounds a primal 6. A cut of capacity 5, equal to the flow, would prove optimality.

Problem 2. Combine Konig with rounding. A bipartite matching of size 2 gives a cover of size 2. On a triangle, rounding returns 3 vertices against integral optimum 2. What is that ratio, and why did Konig not apply?

Answer. The ratio is 3/2. Konig requires a bipartite graph. A triangle is not bipartite, so you use the factor-2 rounding bound instead.